

北京邮电大学

计算机学院（示范性软件学院）

实验报告

课程名称： 计算导论与程序设计

项目名称： 画板程序

项目完成人：

姓名： 侯明玄 学号： 2025211717

指导教师： 崔毅东

日期： 2025 年 12 月 01 日

一. 实验目的

深化学习 C 语言基本语法，从实践开发中了解初等项目开发的具体流程，在动手查阅文档中锻炼个人查询能力，在个人优化代码一步步构建中理解合理命名、合适思路、合规函数的重要性。

二. 实验内容

使用 Raylib 图形库，通过课内知识与图形库 CheatSheet 的运用，制作画板程序。并依据个人喜好增添程序细节。

三. 实验环境

CachyOS(ArchLinux 分支系统) + Code-OSS(VSCode 开源版本[拆分代码时使用 helix]) + Raylib(采用 raylib-quick-start[<https://github.com/raylib-extras/raylib-quickstart>]项目)

四. 附录

4.1. 问题分析

4.1.1. 目标

这个程序意在制作便于使用的画板软件，应当具备完善性、易用性。

4.1.2. 功能

4.1.2.1. 功能一：鼠标绘图相关功能/橡皮擦

当用户的鼠标落在画板上时，如果鼠标左键被按下，则实时读取鼠标位置并画圆，在相邻的两个圆之间绘制直线使得线平滑。绑定 keybinds 调整画笔大小与颜色选择，采用与 canvas 相同的颜色替代橡皮擦。

4.1.2.2. 功能二：绘制/撤销直线与各种图形

通过选择模式的逻辑切换鼠标绘图、直线、三角形、矩形、多边形；当用户切换到对应的图形模式，按照鼠标点击或松开，依次存储点位，并通过 Raylib 内置函数绘制图形[Raylib 的内置函数部分存在缺陷，需要冗杂的手段进行补完]；当用户按下撤销的 Keybinds 时，清除对应模式的最末图形以取消绘制。

4.1.2.3. 功能三：清空画板

通过一个快捷键，当读取到用户按下快捷键后重新绘制画板并清空所有储存的图形；

4.1.2.4. 功能四：提示块/倒计时

提示模块分为启动提示、按下快捷键的总 CheatSheet、操作后提示、与提示小方块四种；总 CheatSheet 通过按下和松开快捷键进行操作；使用 3*fps 的方式设计倒计时模块，在不断刷新 3s 后返回 false 值终结定时提示；

4.1.3. 性能

4.1.3.1. 鼠标绘制的性能

提高 FPS 为 120, 在程序可以有效读取鼠标位置的范围内进行增大使线段更加平滑。

4.1.3.2. 画笔大小的性能

设置界限为 2.0-50.0f, 采用浮点储存, 使调整更精确。

4.1.4. 健壮性

4.1.4.1. 颜色与画笔大小处理

当用户在切换颜色时, 若 colorUsed 的值 exceed 最大颜色数量(列为橡皮擦颜色的颜色除外), 则减小到 0, 反之若小于 0 则增大到最大数量-1; 橡皮擦若减小到设定最小值(2.0f)或增大到最大值(50.0f)则保持不变;

4.1.4.2. 鼠标绘图防断点处理

用户长按鼠标左键绘制的两个圆之间连直线, 使得用户绘制的线更加平滑;

4.1.4.3. Raylib 函数处理

依据 StackOverFlow 与 Github 用户的反馈, Raylib 的 DrawTriangle 函数存在“非一定顺序读入无法绘制”的问题, 采用将结构体内的 point1, point2, point3 换为[2]的数组, 将点顺时针绘制&&逆时针绘制;

4.1.4.4. 矩形绘制处理

矩形在 1, 3 象限中无法绘制, 若 GetMousePosition() 读取到鼠标在此二位置, 则将 point1 与 point2 的 y 坐标互换, 使函数总读入 $x1 < x2$, $y1 < y2$ 的两个点;

4.1.4.5. Shape 统一操作处理

避免某一特定形状在另一形状的上层显示, 采用 Shape 结构体储存其他的结构体类型与对应结构体数组中的位次, 依次绘制/撤销

4.1.4.4. Shape 撤销处理

为避免撤销时出现“-1”等导致后续绘制图形无法显示; 限定撤销必须为 >0 时可以撤销

4.2. 设计方案

拆分为 main.c 与其他负责不同部分的模块, 包括:

- 1.hint 模块, 负责处理提示, 包含 hint.c/hint.h 与用于定时关闭提示的 counter.c/counter.h, hint.c/hint.h 包括各种提示函数, counter.c/counter.h 只需包含一个倒计时函数, 通过 $3 * fps$ 达到延缓 3s 的效果;

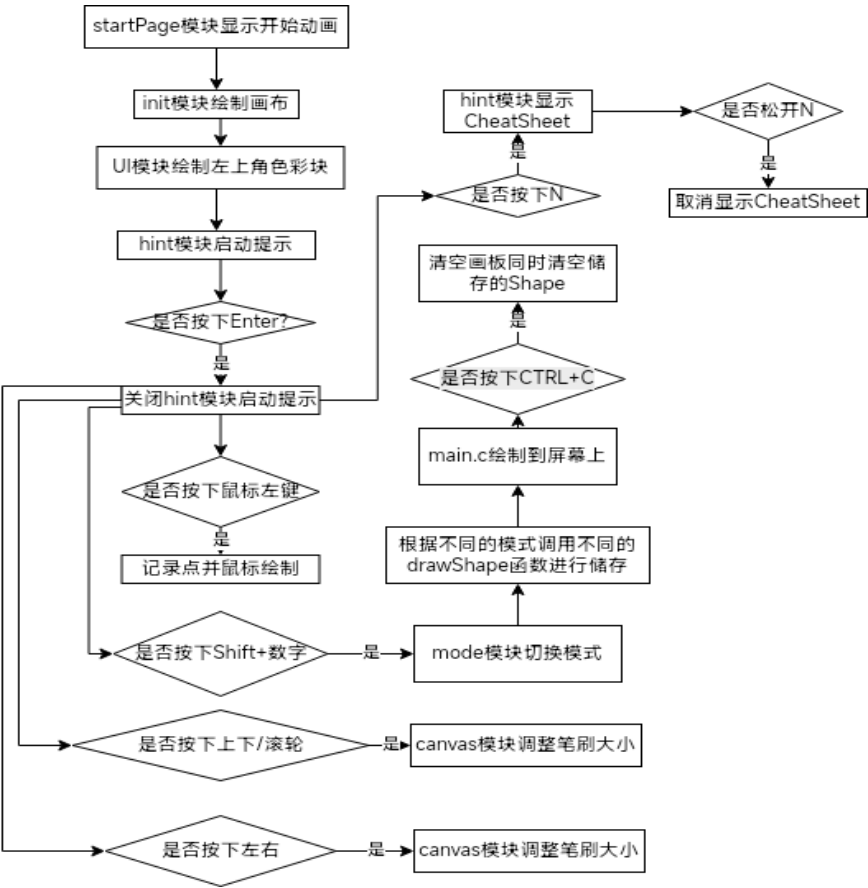
- 2.init 模块, 负责主体内核内容, 包括 canvas.c/canvas.h 与

windows.c/windows.h; 其中 canvas.c/canvas.h 用于处理 canvas 的渲染, 以及鼠标绘制部分。windows.c/windows.h 负责处理程序基本设置, 包括关闭程序, 初始化(包括设置 fps), 以及处理笔刷大小和颜色的函数;

- 3. ui 模块, 负责绘制除画布外的基本 ui, 主要为左上角颜色框;
- 4. startpage 模块, 负责开屏动画;
- 5. mode 模块, 主要为处理模式切换; 以及右上角的 mode 提示框;

—6. Shape 模块, 包括 shape.c/shape.h 与 drawshapec./drawshape.h; 在 shape.h 中包括直线, 三角形, 矩形, 多边形的结构体, 在 shape.c 包括储存图形的函数与在画板上绘制所有图形的函数; drawshape.c 为读取用户绘制图形的基本逻辑, 读取后通过 shape.c 里的储存函数储存在结构体中;

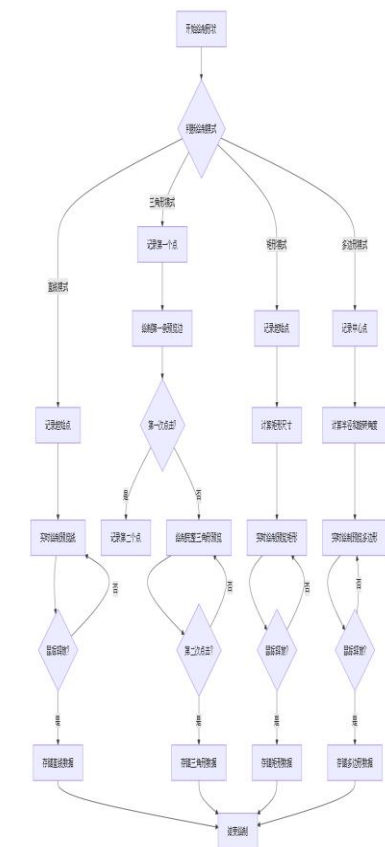
流程图



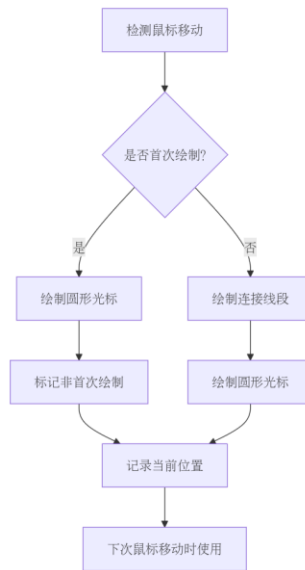
4.3. 重要算法

4.3.1. 算法一：统一形状管理与图形绘制

流程图如下：



流程图如下

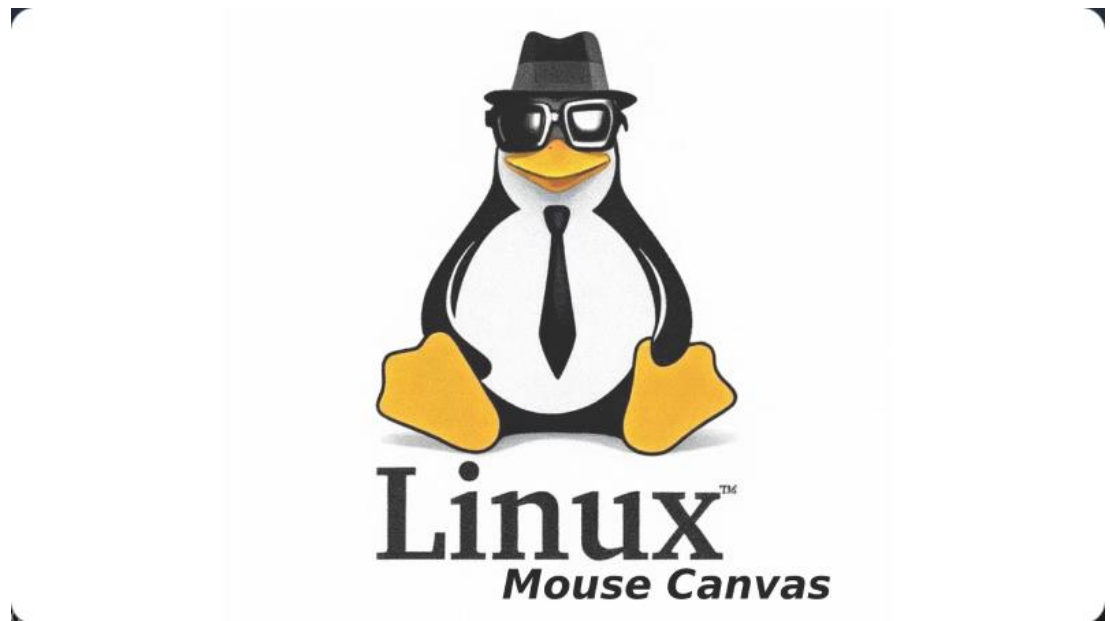


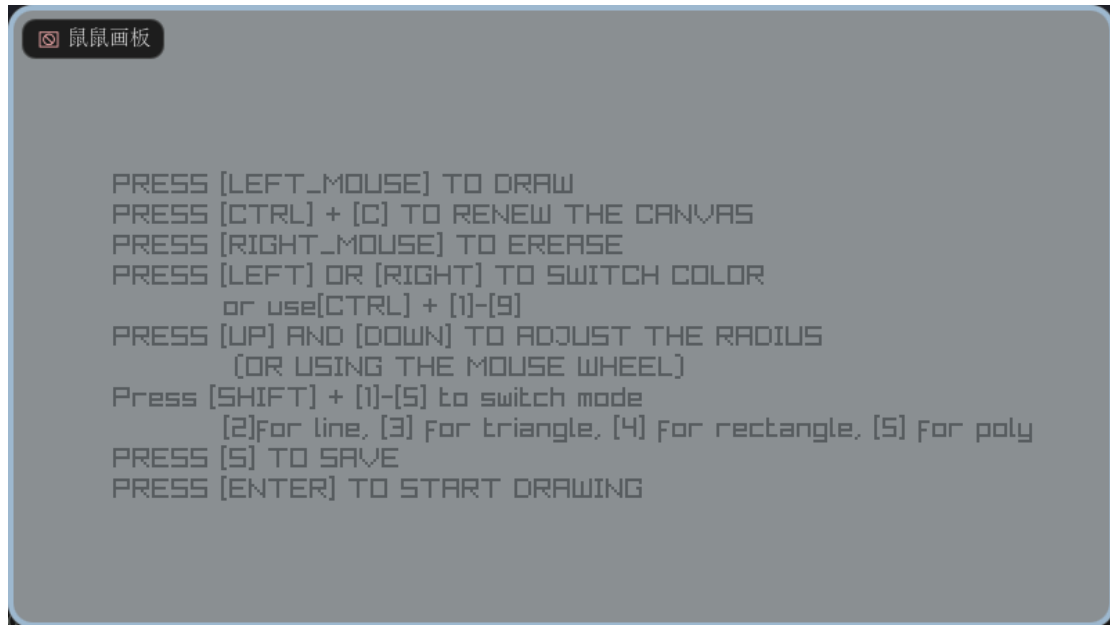
4. 4. 实验结果

*由于本人为 Hyprland 用户，因此侧重于快捷键输入

1. 开屏动画显示

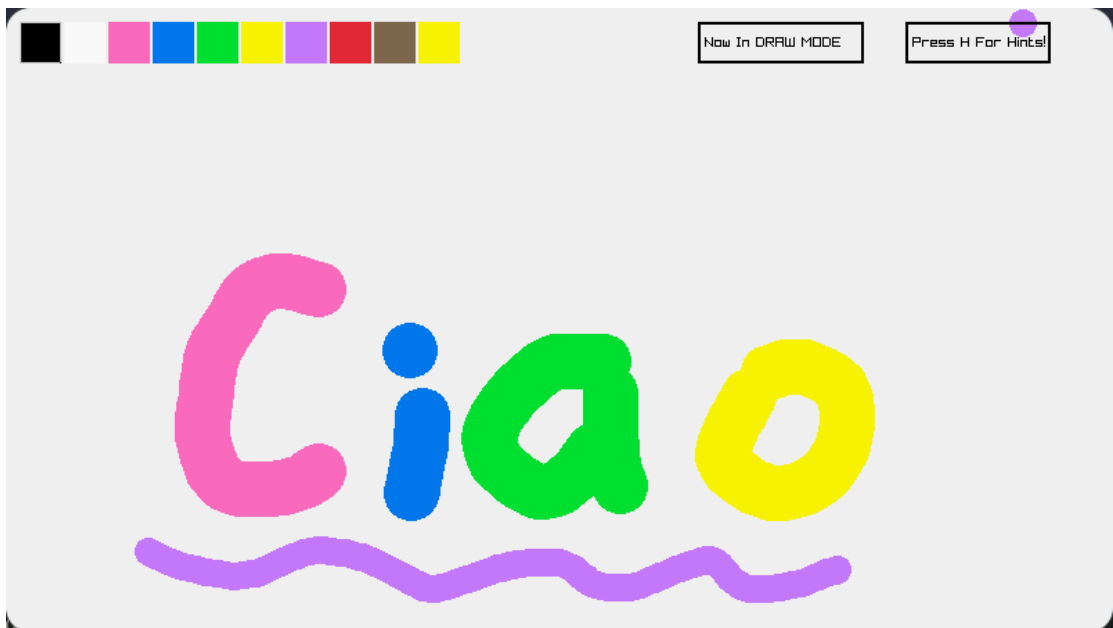
按下 F5 运行项目后，会有 TUX 图案渐入渐出；之后是开屏提示：具体截图如下

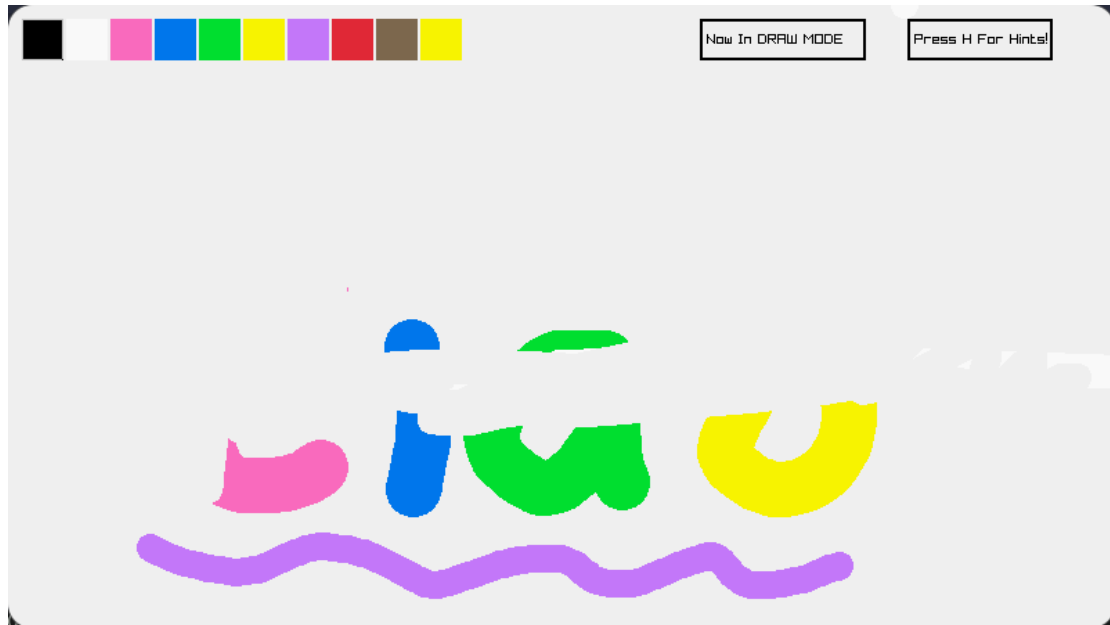




2. 切换颜色&&更改画笔大小&&鼠标绘图

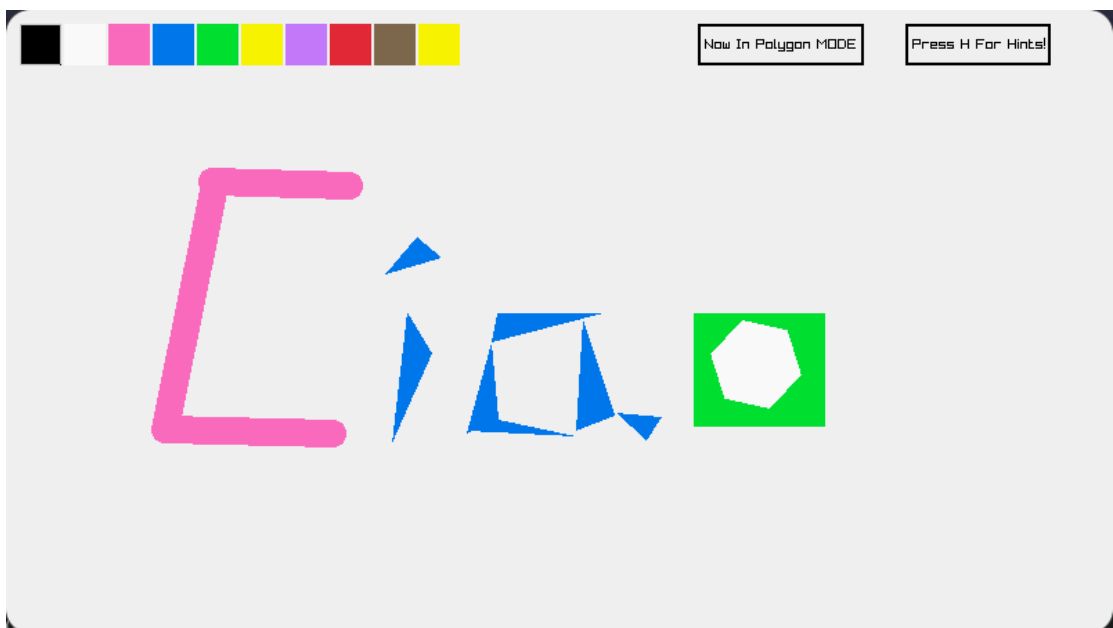
通过[CTRL]+[num]与[左右]进行更换颜色，使用[上][下]与[鼠标滚轮]对画笔大小进行调节，使用鼠标左键绘图，右键充当橡皮擦，具体操作结果如下：

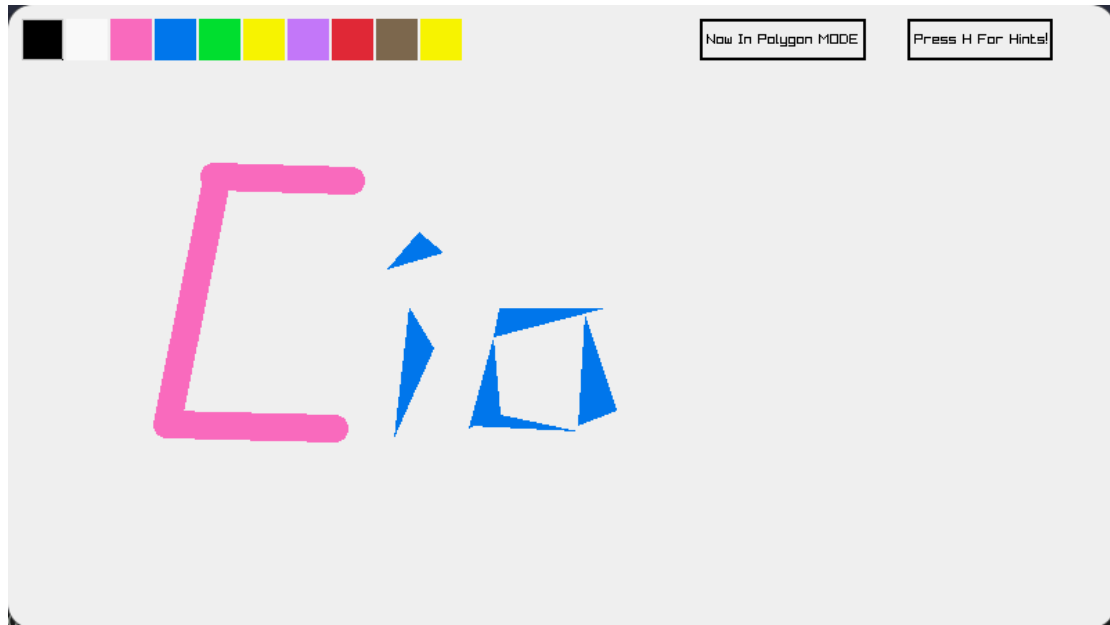




3. 绘制图形&&撤销图形

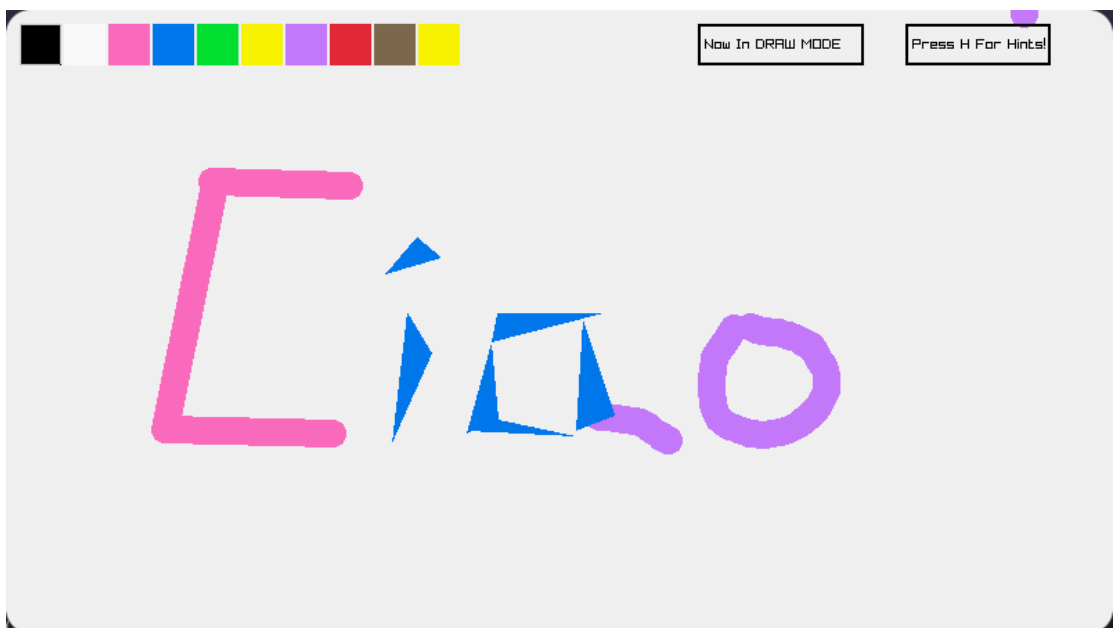
如下为绘制&&[CTRL]+[Z]撤销的结果：





4.清屏

先画出一个`ciao`，之后[CTRL]+[C]效果如下(右上角紫色圆圈为光标)：

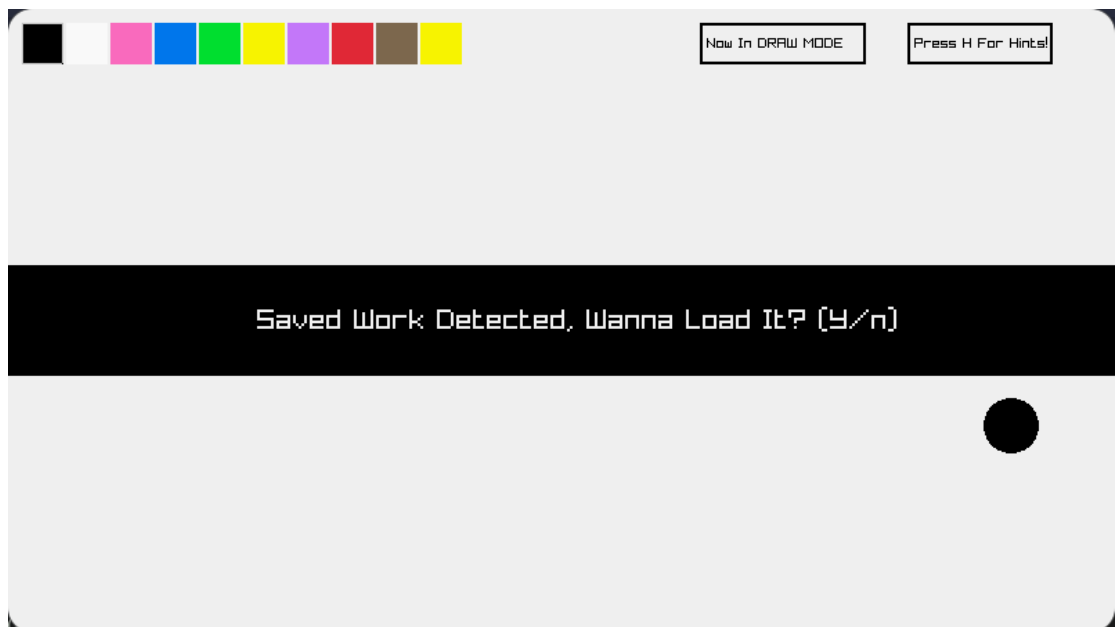
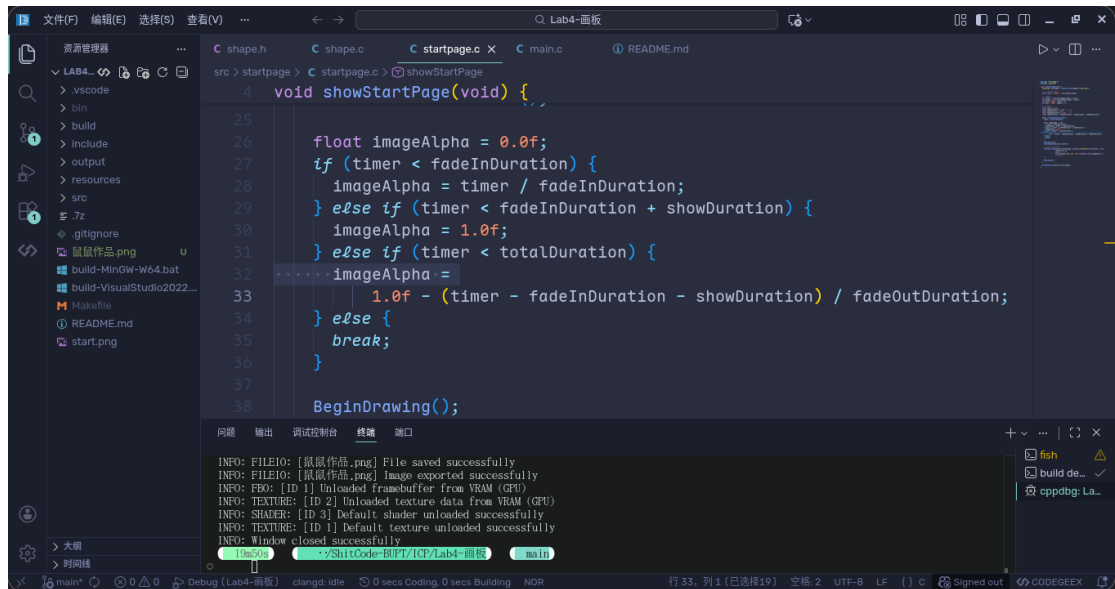




5.保存&&加载

保存后需要再次进入显示提示框，具体操作记录如下





6.其他功能

按[H]显示提示，因为无法在按 H 的时候移动光标的同时截图，因此无法提供截图。

4. 5. 调试心得

4. 5. 1. 遇到设计问题查找 CheatSheet：

寻求 CheatSheet 帮助十分有效，在使用 C 语言一般函数(比如使用 sleep() 函数试图达到提示三秒消失效果发现无法完成的时候)，尝试 Cheatsheet 的函数十分有效。

在开发早期使用 CheatSheet 可以更方便的了解 Raylib 的基本库函数，对于后续的操作十分便捷。

4. 5. 2. 遇到不明问题设置断点单步运行：

在编写项目时进行断点设置、查看变量参数十分有效。尤其是在此种 800 行的大项目的

编写方面。

4.5.3. 遇到非正常问题查找大型论坛：

在此次编写 Shape 部分时，曾遇到“drawTriangle”函数的无法绘制问题，在设置断点后发现所有参数均正常。在寻找 StackOverflow 论坛与 raylib 的 github 仓库 issue 后发现有用用户提出过相同问题，进一步发现函数要求三点输入应保证一定顺序，进一步解决此问题。

4.5.4. 早期编写应当适当拆分：

早期写函数时不注重代码拆分，在 W1 想到一点就加入 main 函数使使用了 21 小时编写了包含 300 行 main 函数的屎山代码，其中包含层层嵌套的用于判断条件的 boolean 量，导致 W2 的代码拆分部分十分不易。

4.5.5. 代码加入注释十分必要：

在编写项目的初期曾不注重注释，导致代码的编写较为混乱，常出现重复变量名等现象。后期加强了注释的引用，并通过给出明显的变量名提升整体可读性。